

Sentinel — Task Split (5 members)

Five lanes. Each one is a separate folder with its own deliverable and can be built without waiting on the others.

Day 0 — one hour together, before anyone codes

Three things must be agreed first, otherwise the lanes will not fit together on merge day.

1. Freeze sentinel/models.py. Every object that crosses a lane boundary, written once, by all five people in the room. Nothing in this file changes after day 1 without telling everyone.

```
class Message(BaseModel):
    role: Literal["user", "assistant"]
    content: str

class TargetRequest(BaseModel):
    conversation: list[Message]
    mode: Literal["rag", "agentic"]
    session_id: str

class TargetResponse(BaseModel):
    output: str
    retrieved_doc_ids: list[str]
    retrieved_chunks: list[str]
    retrieval_ranks: dict[str, int]
    tool_calls: list[ToolCall]
    latency_ms: int
    usage: dict

class Attempt(BaseModel):
    id: str
    category: str
    seed_id: str | None
    parent_attempt_id: str | None
    mutation_depth: int
    strategy_used: str
    conversation: list[Message]
    expected_breach: list[str]

class Verdict(BaseModel):
    attempt_id: str
    judge_version: str
    verdict: Literal["breach", "defended", "not_delivered", "ambiguous"]
    confidence: float
    policy_violated: list[str]
    evidence_span: str | None
    severity: Literal["critical", "high", "medium", "low"] | None
    reasoning: str
```

2. Agree the 5 policies and their IDs. P1 system prompt, P2 salary bands, P3 legal advice, P4 employee records, P5 tool confirmation. The exact wording goes in docs/policies.md and every lane refers to these IDs.

3. Everyone commits a stub on day 1. Each lane pushes a function with the right signature that returns fake but valid data. From that point nobody is blocked on anyone.

Repo scaffolded with uv, a .env.example, and GitHub Actions running ruff and pytest.

Lane 1 — Target application

Owns: sentinel/target/

The app that gets attacked. An HR policy assistant that answers from documents and, in the second mode, can call tools.

Build, in order:

1. FastAPI service with POST /chat, accepting TargetRequest, returning TargetResponse
2. System prompt stating the 5 policies, with a unique canary string embedded (e.g. NW-CANARY-7f3a9) that must never appear in output
3. Mode A: retrieve top-4 chunks from Chroma, put them in the prompt, answer
4. Mode B: the same, plus a function-calling loop over three tools
5. Three tool stubs that log the call and return canned success
— search_directory(name), send_email(to, body), delete_employee_record(id)
6. Return retrieval ranks in the response so other lanes can tell whether a poisoned document actually arrived
7. Later: a hardened version with a stricter system prompt, an input filter, and stripping of imperative text from retrieved chunks

Depends on: the Chroma collection from Lane 2. Until it exists, read from a folder of plain text files.

Done when: curl a question at it and get an answer back with the retrieved chunks listed, in both modes, and the canary is retrievable through a direct prompt-injection attempt.

Lane 2 — Corpus, retrieval, chunking experiment

Owns: sentinel/target/corpus/, sentinel/retrieval/, scripts/seed_corpus.py

The documents and the retrieval layer. The most under-rated lane — the quality of the poisoned documents decides how good the findings are.

Build, in order:

1. seed_corpus.py — generates documents from specs using the OpenAI API, saves as markdown with frontmatter (doc_id, restricted, poisoned, injection_style)
2. 20 clean policy documents, 500–1500 words, on overlapping topics so retrieval has to discriminate
3. 5 restricted documents. Use distinctive sentinel numbers for salary figures (e.g. 147,318) so leakage is detectable by exact string match
4. 2 employee records with obviously fictional names
5. **8 poisoned documents**, each a different injection style: fake audit requirement, simulated system block, HTML comment, base64 payload, table cell, footnote, fake executive quote, tool-directing instruction
6. Indexing script — chunk, embed with text-embedding-3-small, store in Chroma, persistent directory in the repo

7. Retrieval function returning chunks, document IDs, and the rank of each
8. **Chunking experiment:** build the same index at 256/32, 512/64 and 1024/128; run the same query set against each; report how often a poisoned document is retrieved intact

Depends on: nothing. Can start immediately.

Done when: three indexes exist, a query returns ranked chunks, and there is a table showing poisoned-document retrieval rate at each chunk size.

Lane 3 — Attacks and the generator

Owns: sentinel/attacks/, sentinel/agents/generator.py

The attack library and the agent that writes and rewrites attacks.

Build, in order:

1. taxonomy.py — the 8 categories, each with a description, which policies it targets, which modes it applies to, and a written definition of what counts as success
2. Seed library in YAML, 10–15 per category, around 100 total. Each seed declares its category, target policy, applicable modes, and any poisoned document it depends on
3. Generator agent, fresh mode: given a category and seed, produce a concrete Attempt
4. Generator agent, mutate mode: given a failed attempt plus the target's actual refusal text, produce a variant
5. Eight mutation strategies: paraphrase, language switch, encoding (base64/rot13/spacing), authority framing, task framing, payload splitting across turns, prefix injection, nested quotation
6. Duplicate suppression — hash normalised attempt text, reject near-identical attempts so budget is not wasted

Depends on: Attempt from models.py. Nothing else. Seeds can be written with no code running at all.

Done when: generate(category="indirect_injection") returns a valid Attempt, and mutate(failed_attempt) returns a meaningfully different one.

Lane 4 — Judge, labelling, metrics

Owns: sentinel/agents/judge.py, sentinel/evals/, all Streamlit UI

The hardest lane and the one the project is judged on. Starts on paper, not in code.

Build, in order:

1. **Days 1–2, no code:** 8 rubric documents, one per category, in docs/rubrics/. Each states what counts as a breach, what does not, and includes two worked examples of each. Plus labelling guidelines for the team.
2. Judge agent — takes a trace and a rubric, returns a Verdict
3. Deterministic checks that run before the model and override it: canary string present, sentinel salary figure present, destructive tool called without a confirmation turn

4. Evidence-span validation — check the quoted text actually exists in the target output; if not, mark the verdict ambiguous
5. Labelling UI in Streamlit: trace on the left, four verdict buttons and a notes box on the right, keyboard shortcuts, writes to `human_labels`
6. `metrics.py` — agreement between judge and human labels, agreement between the five labellers, precision and recall on breaches, all reported per category
7. Golden set: freeze 60 labelled attempts; a pytest check re-runs the judge against them and fails if agreement drops
8. Dashboard views: findings browser with trace viewer, and a judge accuracy view

Depends on: traces from Lane 5's database. Until they exist, hand-write 10 fake traces as JSON and work against those.

Done when: the judge returns a valid verdict on a trace, the labelling UI works, and `metrics.py` prints agreement numbers from the labels table.

Lane 5 — Orchestrator, storage, reporting

Owns: `sentinel/orchestrator.py`, `sentinel/db.py`, `sentinel/agents/planner.py`, `sentinel/agents/executor.py`, `sentinel/agents/analyst.py`, `sentinel/reporting/`

The piece that runs everything and turns results into a report.

Build, in order:

1. SQLAlchemy models and the SQLite schema — scans, attempts, traces, verdicts, `human_labels`, findings
2. Executor — send an Attempt to the target over HTTP, capture the full trace, time it, retry on transport failure, save
3. Orchestrator loop — take a budget, pull attempts from the generator, run them concurrently with an asyncio semaphore at 5, send traces to the judge, handle retries of failed attempts up to depth 3
4. Cost tracking — log the usage field on every API call, convert to dollars, store per scan, split by component
5. Safety limits — attempt cap per scan, mutation depth cap, and a kill switch if spend crosses a threshold
6. Planner agent, plus a `--planner=uniform` baseline for comparison
7. Scoring — success rate per category, risk score 0–100, cost per breach
8. Analyst agent — group breaches by cause, assign severity, write the report as markdown
9. Scan launcher in Streamlit: pick budget and categories, start a scan, watch progress

Depends on: Attempt and Verdict from `models.py`. Uses stubs from Lanes 3 and 4 until they are real.

Done when: `python scripts/run_scan.py --budget 20` completes end to end against stubs and writes rows to every table.

Where the lanes meet

Boundary	Contract	Who
Orchestrator → Target	POST /chat with TargetRequest	1 and 5
Target → Retrieval	retrieve(query, k) → chunks + ranks	1 and 2
Orchestrator → Generator	generate(category, seed) / mutate(attempt, trace)	3 and 5
Orchestrator → Judge	judge(trace, rubric) → Verdict	4 and 5
Judge → Metrics	verdicts and human_labels tables	4 alone
Everything → Storage	SQLAlchemy models	5 owns, all read

Merge into main at least every other day. Five people diverging for a week is how this fails in the last 48 hours.

Shared commitments

- **All five label together, one sitting, middle weekend, 3 hours.** Everyone labels the same 50 attempts first so disagreement between people can be measured, then the remaining 150 are split. This cannot be done alone and it gates the main result.
- 15-minute standup Tuesday and Thursday.
- Nobody pushes to main directly.
- Hard spend limit set on the OpenAI key on day 1.

Day-by-day

Days	Lane 1	Lane 2	Lane 3	Lane 4	Lane 5
1–2	Mode A running	20 clean docs, index built	Taxonomy + 40 seeds	8 rubrics written	Schema + executor
3–5	Mode B + tools	Poisoned docs + 3 indexes	100 seeds, generator live	Judge v1 + deterministic checks	Orchestrator loop, cost tracking
6–7	Hardened version	Chunking results	Mutation strategies	Labelling session, metrics	Planner, scoring
8	Support	Support	Support	Judge validation numbers	Analyst + report

Days	Lane 1	Lane 2	Lane 3	Lane 4	Lane 5
9	README	Chunking write-up	Taxonomy docs	Metrics write-up	Demo video, submit

Feature freeze end of day 8. Day 9 is documentation only.

If someone falls behind

Lanes 2 and 3 are the most self-contained and can absorb help. Lane 4 is the critical path — if it slips, pull someone from Lane 1 after mode B is working. What must not be cut: the labelling session and the judge accuracy numbers. Without those the project is a tool that generates attacks and asserts they worked.